PRODUCT + PITCH + DOCUMENTATION MASTER BRIEF

AccessPath

A journey-based accessibility audit for the live web

For the teammate preparing the AZINHACK 2026 presentation and repository documentation while product development proceeds in parallel.

> **Status on 17 October 2026:** AccessPath is a selected planning direction, not yet a verified working build. All implementation claims, screenshots, links, metrics, and demo outcomes must be updated from the actual repository before submission. An older Resolve prototype exists separately and is not the AccessPath project.

Submission	Date / requirement
Online	18 Oct 2026, 11:59 PM IST - registration, PPT and GitHub repository
On site	21 Oct 2026, 09:30 AM IST mentorship; 22 Oct 2026, 10:00 AM IST finale
Core rule	Open innovation; meaningful, functional TinyFish integration

Use this brief as the single source for the proposed narrative. The code repository and tested product become the source of truth for completed features.

01 Executive snapshot

One sentence: AccessPath follows a real user goal across a public website, audits the pages encountered, and gives the team a reproducible, evidence-backed list of accessibility issues and items needing human review.

The problem: teams often audit a single URL or rely on a generic score. People experience an entire journey: finding information, moving between pages, and completing a task. A page can pass many automated checks while the journey remains difficult or impossible.

The product: enter a public HTTPS URL and a task such as 'find the contact form.' TinyFish Web Agent explores the route. A bounded browser audit uses Playwright and axe-core on the encountered pages. The report displays the journey, affected elements, evidence, and suggested next checks.

Why TinyFish matters: goal-driven navigation discovers a task-specific route on the live web. TinyFish Browser API then provides an isolated cloud browser via CDP for page audit and screenshot evidence. Removing TinyFish would remove the product's journey-discovery layer.

Who benefits: developers and QA teams, small agencies, university or public-service web teams, and product owners who need to prioritize fixes by the task a user tried to complete.

Pitch language to use

> 'AccessPath turns an accessibility scan into a user-journey investigation. Tell it what a visitor is trying to do; it follows the path, tests what it can verify, and returns evidence a developer can act on.'

Why it could stand out

- Journey context gives a credible product distinction from a one-page automated audit.
- TinyFish is an actual execution dependency rather than a token mention in the architecture slide.
- A broken-to-fixed controlled demo can show objective before/after evidence.
- The system has a concrete path from hackathon MVP to a multi-user service.

Claims policy

Say 'designed for multiple users' until multi-user auth, isolation, queueing and load behavior are verified. Say 'automated findings' rather than 'WCAG certified.' Do not invent accuracy, time savings, users, uptime, cost, or scan throughput. Mark future features as roadmap items.

02 Event facts and submission obligations

The AZINHACK 2026 listing describes a 2-day event at USAR, GGSIPU East Delhi Campus, Delhi, an open innovation track, team size 1-4, and a meaningful TinyFish integration requirement. It gives the online deadline and the two on-site checkpoints below. The event listing should be rechecked immediately before upload because organizers can update logistics.

Item	What the teammate must prepare
PPT	Problem; proposed solution; target users/use case; AI or ML role; technical approach; TinyFish integration; expected impact; feasibility; future scope.
GitHub	Project code with a clear README, setup instructions, and implementation details. Use the actual AccessPath repository, not the older Resolve prototype.
Profiles	Team members' GitHub, LinkedIn and X profiles are requested in the profile submission stage. Get exact URLs from members; do not invent them.
Deadline	18 Oct 2026 at 11:59 PM IST for registration and online project submission.
On-site	21 Oct 09:30 mentorship; 22 Oct 10:00 finale. Verify any later communication from organizers.

Source of record

AZINHACK 2026 Unstop listing (accessed 17 Oct 2026). The original public listing URL is also usable: Unstop event page.

Do not import rules from other hackathons

This brief is for AZINHACK 2026 at GGSIPU. Similar TinyFish events have different round-one requirements. Avoid copying a template, slide limit, or MVP rule from another listing.

03 Product specification and scope

Core user flow

Step	Visible user experience	System behavior
1. Start	Sign in; enter public HTTPS URL and task.	Validate account, URL, quota and request key.
2. Queue	See accepted scan with clear pending state.	Persist job; return scan ID quickly.
3. Explore	See progress and discovered route.	TinyFish Web Agent follows goal and returns structured page URLs.
4. Audit	See page progress, then findings.	TinyFish Browser session + Playwright run axe-core; screenshot each allowed page.
5. Review	Read timeline, issues, evidence, manual checks.	Store normalized findings and private evidence; show owner-authorized result.

MVP in scope

- One public HTTPS starting URL and one short task per scan; at most five pages and one browser session.
- A journey timeline showing each audited URL and page status.
- Rule ID, impact, selector or element reference, description, help link, screenshot, and review status.
- Queued, running, completed, partial, failed and cancelled states with understandable messages.
- Owner-only scan history and evidence access once auth is implemented.

Explicitly out of MVP

Authenticated target-site flows, payments, checkout, form submission, private networks, downloaded files, sweeping site crawls, automatic remediation, full WCAG conformance certification, and a fabricated aggregate accessibility score. These may appear only as future opportunities when appropriate.

Example input and output

Input: URL https://demo.example.org ; task 'Find the contact form.'
Output: Home > Contact path; two deterministic automated findings such as an unlabeled input and a missing text alternative; screenshot evidence; a note that keyboard flow and usability need human review. These are illustrative examples, not measured AccessPath output.

04 TinyFish and audit pipeline

The proposed implementation uses two different TinyFish capabilities. The Agent API performs task-driven navigation and returns a structured route. The Browser API creates a separate isolated browser with a CDP URL. Playwright attaches to that browser to audit the selected pages. The two sessions do not automatically share page state; the worker reopens allowed discovered URLs. Dynamic state may not replay perfectly and must be treated as a limitation.

Stage	Proposed contract	Guardrail / failure
Agent run	POST /v1/automation/run-async with URL, goal and output schema; poll GET /v1/runs/{id}.	Do not rely on beta-only max_steps; bound wall time; cancel async run if needed.
Route	Parse URLs and action summaries from structured result; validate each URL again.	Reject non-public, cross-policy or excessive targets; report partial discovery.
Browser	POST Browser API session; connect Playwright to returned cdp_url; DELETE session.	Session creation may take 10-30 seconds; always close in finally, retry cleanup when needed.
Audit	Navigate up to five approved pages; inject axe-core; capture screenshot and page metadata.	Cap page time, pixels and evidence size; page errors do not erase other findings.
Report	Normalize raw rule findings separately from explanatory copy.	Never recast a failed or incomplete check as a pass.

AI/ML explanation for PPT

TinyFish Web Agent is the AI-driven component: it interprets a natural-language goal and navigates a live website. axe-core is a deterministic rules engine, not a custom ML model. AccessPath adds orchestration, policy checks, storage, evidence and a useful report. Do not claim an original neural network or model training.

Truthful accessibility language

Automated checks can identify some problems and give repeatable technical evidence. W3C states that no tool alone determines whether a site is accessible. The interface must surface manual-review items and avoid a pass/fail certification claim.

References: TinyFish Agent API; TinyFish Browser API; axe-core; W3C evaluation overview.

05 Technical architecture and stack

Browser UI -> **Next.js API on Vercel** -> **Supabase Auth/Postgres** -> **private Railway worker** -> **TinyFish Agent + Browser** -> **axe-core findings + Supabase private evidence** -> **owner-only report**

Layer	Exact choice	Reason / boundary
Workspace	TypeScript + pnpm monorepo	Shared request/response types, one lockfile.
Web	Next.js App Router + React + CSS Modules + Zod	Fast forms/report UI, short API handlers, server-side validation.
Hosting	Vercel	Web deployment and edge protections. Do not run browser scans in a request.
Identity/data	Supabase Auth + PostgreSQL + RLS	Account isolation, durable jobs and findings.
Evidence	Private Supabase Storage	Screenshots are not world-readable; owner-authorized signed access.
Worker	Node.js 24 service on Railway	Long-running jobs, bounded browser concurrency; no public worker API.
Queue	Postgres row claims with short leases and SKIP LOCKED	Safe multi-worker claiming without Redis in MVP.
Live web	TinyFish Agent API + Browser API + Playwright CDP	Goal navigation and isolated execution.
Checks	axe-core	Reproducible rule IDs and affected nodes.
Quality	Vitest, Playwright, GitHub Actions	Logic, end-to-end fixture, typecheck and build.

Scale path, stated precisely

The architecture separates short user requests from long-running scans. Indexed Postgres claims can distribute jobs across workers; limits and cost budgets bound work. Scale beyond the initial worker is a design path, not a validated performance result. The first launch settings are one active scan per user, five scans per user per day, one active scan per target domain, four browser sessions globally, plus a global daily cap.

06 Data model and API contract

Table	Minimum fields / role
scans	id, user_id, target_url, task, status, attempts, lease_until, created_at, finished_at, error_code, idempotency_key; indexed for owner history and queue claims.
scan_steps	id, scan_id, position, url, title, action_summary, state; ordered route.
findings	id, scan_id, step_id, rule_id, impact, selector, description, help_url, review_status; preserve source rule.
artifacts	id, scan_id, step_id, storage_path, kind, created_at; private evidence metadata.
usage_daily	user_id, day, scan_count; atomic quota counter.

Public routes

Route	Response / behavior
POST /api/scans	Authenticated body {url, task}; Idempotency-Key header; 202 {id,status:'queued'} after validation and quota check.
GET /api/scans?cursor=...	Paginated scans owned by caller.
GET /api/scans/:id	Owner-only scan, steps, findings and short-lived evidence access.
POST /api/scans/:id/cancel	Owner-only, repeat-safe best-effort cancellation for queued/running work.
GET /api/health	Basic web health; worker heartbeat is monitored privately.

State semantics and contract notes

queued -> running -> completed. Terminal alternatives: partial, failed, cancelled. A page-level failure can yield a partial report if other pages succeeded. Duplicate requests with the same owner and idempotency key must return the same scan. Pagination must be owner-filtered at both query and RLS levels. The report must not display an unsigned raw storage path as a public URL.

07 Security, DDoS, privacy and cost controls

Threat / load	Planned control	Verification before claim
Public API flood	Vercel edge mitigation and WAF; authenticated POST; payload cap; quota and 429 response.	Exercise quota and rejection; inspect deployment settings. Never claim immunity from DDoS.
SSRF	HTTPS/public DNS/IP allowlist policy; reject localhost, private/link-local, metadata IPs, credentials and unusual ports; revalidate redirects.	Test URL parsing, DNS resolution and redirect cases.
Paid resource exhaustion	One active/user, five/day/user, one active/domain, four sessions global, daily spend cap; time/page/image limits.	Atomic counters; concurrent request tests; monitor TinyFish cost.
Cross-user leakage	RLS on all owner records; private storage; short-lived authorized evidence links.	Two-user access tests for read, list, cancel and storage.
Queue duplication	Idempotency key, lease expiry, SKIP LOCKED, bounded retry and dead-letter failure.	Concurrent claim test; restart recovery.
External content	Treat page text and agent output as untrusted; do not follow page instructions; sanitize UI text.	Prompt-injection and XSS fixtures.
Secrets and logs	Server-only TinyFish and service-role keys; redact target page content and tokens; screenshot retention policy.	Search client bundle/repo; inspect log examples and deletion job.

User-facing boundaries

No target-site login, checkout, payment or form submission in the first release. Audits operate only on public pages. A user should scan sites they are authorized to inspect. Keep sample target controlled by the team for the judged demonstration.

Deployment detail

Use Vercel for the web tier and a Railway worker without a public endpoint. Keep Supabase service credentials only server-side. Set database RLS and private storage policies before accepting real users. Attach alerts for queue depth, 429 volume, failures, browser cleanup and TinyFish spend.

References: Vercel DDoS mitigation; Supabase RLS; Supabase Storage access control; Railway private networking.

08 Frontend and visual language

The interface should feel like a serious editorial product for engineering teams: warm off-white canvas, deep ink type, one restrained teal accent, generous whitespace, fine dividers, and clear evidence hierarchy. Avoid gradient hero banners, decorative dashboard charts, generic AI orb graphics, and unverified scores.

Screen	Required content / behavior
Landing	One-line value proposition; concrete task-to-report example; short TinyFish explanation; simple call to action; honest accessibility caveat.
New scan	Labeled URL and task fields; examples; policy warning; live validation; queue status; clear quota errors.
Dashboard	Recent scans with status, task, domain and time; empty, loading, failed states; cursor pagination.
Report	Journey timeline; page status; findings grouped by impact; screenshot beside element and rule explanation; manual-review section; partial failure context.

Suggested tokens and type

Typography: Instrument Sans for interface, IBM Plex Mono for URLs, rule IDs and small metadata if available; system fallbacks are acceptable. Palette direction: ink #142C30, teal #087E76, paper #F8F6F0, pale teal #EAF4F0. These are design targets, not yet implemented assets.

Accessibility of the product itself

- Semantic headings, explicit field labels and programmatic error association.
- Visible focus, keyboard-operable timeline and evidence controls.
- Readable contrast, reduced motion, responsive layout, no color-only status coding.
- Announce loading/failed states and preserve text explanation when screenshots are unavailable.

PPT visual system

Use the same ink/teal/paper palette and one large visual per slide. Show a real flow diagram, a real product screen when it exists, and a close crop of an actual finding. Until the build produces screenshots, mark visual mockups 'concept UI' directly on the slide.

09 Repository blueprint and file ownership

This is the planned AccessPath repository. The teammate may create or update README/docs/PPT in the new AccessPath repo after it exists, while developers own application code. Avoid editing the read-only synced project sources or using Resolve files as AccessPath proof.

```
accesspath/
  README.md                .env.example            .gitignore
  LICENSE                  package.json             pnpm-workspace.yaml
  pnpm-lock.yaml
  apps/
    web/
      package.json          next.config.ts
      src/app/
        page.tsx             login/page.tsx
        dashboard/page.tsx   scans/new/page.tsx
        scans/[id]/page.tsx  api/scans/route.ts
        api/scans/[id]/route.ts
        api/scans/[id]/cancel/route.ts
        globals.css
      src/components/
        scan-form.tsx        scan-list.tsx
        journey-timeline.tsx finding-list.tsx
        evidence-viewer.tsx
      src/lib/
        supabase-browser.ts supabase-server.ts
        quotas.ts           url-policy.ts
      src/proxy.ts
    worker/
      package.json          Dockerfile
      src/index.ts          src/claim.ts
      src/tinyfish-agent.ts src/tinyfish-browser.ts
      src/audit.ts          src/evidence.ts
      src/report.ts
  packages/contracts/src/index.ts
  supabase/migrations/001_initial.sql
  supabase/tests/rls.sql
  demo-site/                tests/
  docs/
    ARCHITECTURE.md         SECURITY.md
    DEMO.md                 PITCH.md
    AccessPath-Pitch.pptx
  .github/workflows/ci.yml
```

Collaboration rule

Slides and docs can be drafted immediately from this brief. Every feature status, command, secret name, screenshot and deployment URL must be reconciled with the current code before final commit. The development team should provide a short status note at each gate; the documentation teammate updates the claims ledger and deck in the same pass.

10 Development gates and proof required

Gate	Build target	Proof to share with PPT/docs owner
A - skeleton	Repo, workspace, shared schemas, env example, README, controlled demo site.	GitHub URL, install command, app boot screenshot, demo URL/task.
B - TinyFish	Real Agent run; Browser session; Playwright CDP; clean closure.	Run ID (safe to share), visited URL, real screenshot, exact API call path.
C - evidence	axe-core on up to five pages; findings and screenshot normalization.	Two known defects detected; fixed version removes same findings; raw rule IDs.
D - multi-user	Auth, migration/RLS, queue leases, API, private evidence.	Two-account isolation result; idempotency result; secrets absent from client.
E - polish	Landing/form/dashboard/report; mobile/keyboard QA; errors, limits and telemetry.	Final product screenshots, one mobile image, status-state list.
F - ship	CI; Vercel/Railway/Supabase; real production scan; final docs and slides.	Live URL, commit SHA, passing checks, production TinyFish result, submission links.

Time and priority

The roadmap estimated Gates A-F at roughly 7-10 hours, as planning ranges only. Protect the working TinyFish path, evidence loop, owner isolation and final deployment/submission. Defer export formats, broad integrations and invented AI scoring if time is tight.

Acceptance criteria for a defensible demo

- User submits one task and receives a scan ID without waiting for a browser in the HTTP request.
- TinyFish performs a real goal-based run and returns an auditable route.
- At least two known defects have rule IDs, affected elements and evidence; corrected fixture removes them.
- A second account cannot access another account's scan or evidence.
- Failed/partial scans are understandable; session cleanup and quotas are visible in logs/tests.
- Final PPT and README match the exact deployed feature set.

11 PPT production brief - slides 1 to 6

Target 10-12 concise slides unless Unstop or organizers specify a template or limit. Keep editable PPTX and export a PDF only if the submission form requests one. One claim and one strong visual per slide.

Slide	Headline + exact content	Visual / presenter cue
1	AccessPath: accessibility along the journey, not just one page. Team names and AZINHACK 2026.	Product screen if real; otherwise marked concept mockup. 10-second framing.
2	Problem: a user task spans multiple pages; isolated automated scans miss route context. Accessibility problems block people from completing tasks.	Simple 3-step journey with failure point. Avoid uncited market statistics.
3	User and use case: developer/QA team checks 'find the contact form' on a public site; receives actionable evidence.	Persona + input/output flow.
4	Solution: enter URL + goal; TinyFish explores; axe-core checks each allowed page; AccessPath reports the path and issues.	Four-stage product flow. Label each technology correctly.
5	TinyFish integration: Agent API for goal-driven discovery; Browser API/CDP for isolated audit. Show real API proof once Gate B passes.	Architecture zoom; run ID and screenshot only after verified.
6	AI/ML role: TinyFish interprets goal and navigates; axe-core provides deterministic findings; human review handles what automation cannot.	Three-column 'AI / rules / human' diagram.

Speaker notes for slides 1-6

Open with a concrete task, not a sweeping claim about all websites. Explain that the TinyFish agent discovers where a user goes and that the browser audit is separately bounded. When asked about ML, explicitly say the team is integrating an agent rather than training a model. Cite W3C for the human-review boundary.

12 PPT production brief - slides 7 to 12

Slide	Headline + exact content	Visual / presenter cue
7	Technical approach: Next.js/Vercel web; Supabase Auth/Postgres/RLS/Storage; Railway worker; TinyFish; Playwright + axe-core.	Single architecture diagram with arrows and trust boundaries.
8	Live product or controlled demonstration: broken sample journey; two findings; corrected sample; rescan.	Before/after evidence with rule IDs. If not built, title as planned demo.
9	Scalability and safeguards: short API, durable queue, bounded workers, per-user/domain limits, RLS, private evidence, WAF.	Lifecycle plus 4 controls. State as implemented only after verified.
10	Impact and feasibility: faster prioritization hypothesis; narrow public-site MVP; measurable acceptance gates.	Show actual findings or a 'measurement plan', never fabricated impact numbers.
11	Future scope: manual review workflow, issue ownership, report export, permitted broader site coverage, measured scaling.	Three-stage roadmap: MVP / onsite / later.
12	Close: real journey, real evidence, real TinyFish use. GitHub and live demo URLs once valid; team names.	One clean product image and QR codes only for verified URLs.

Deck-level quality gate

- Every event-required topic appears by slide 11: problem, solution, users, AI, tech, TinyFish, impact, feasibility and future.
- Screenshots must come from the current AccessPath build; label conceptual visuals.
- No tiny body text, pasted code walls, decorative charts, or more than one key idea per slide.
- Speaker notes contain a 2-minute demo narrative and candid answers on false negatives, privacy, scale and cost.
- Final slide links and QR codes are opened once in a clean browser before submission.

13 README and documentation writing brief

README.md - required sections in order

1. Product name and one-sentence summary. 2. Current implementation status and live links. 3. Problem and example user journey. 4. What the current build actually does. 5. TinyFish integration with Agent/Browser distinction. 6. Architecture diagram. 7. Stack. 8. Prerequisites and exact local setup commands. 9. Environment variables by service with safe placeholders. 10. Database migration and RLS setup. 11. Running web, worker and controlled demo. 12. Testing. 13. API routes. 14. Security/usage limits and current gaps. 15. Demo walkthrough. 16. Roadmap. 17. Team, licensing and source references.

docs/ARCHITECTURE.md

Include trust-boundary diagram, request/worker flow, queue claim and lease rules, TinyFish lifecycle, data model, API shapes, evidence storage, failure states, and why long browser jobs run outside Vercel requests. Add measured bottlenecks only after testing.

docs/SECURITY.md

Document URL/redirect/DNS policy, SSRF limits, auth/RLS, storage policies, secrets, scan quotas, session cleanup, log redaction, retention, incident response contact, and known limitations. Distinguish controls shipped from planned ones with a dated checklist.

docs/DEMO.md

List the exact controlled site URL and goal; environment prerequisites; expected broken findings and rule IDs from the actual run; before/after sequence; fallback screenshots; failure recovery; demo duration; and which evidence is synthetic fixture content.

docs/PITCH.md

Keep the problem, solution, target users, differentiation, TinyFish necessity, truthful AI description, feasibility, impact hypothesis, limitations and roadmap in prose. Link to primary sources. Keep deck and README wording aligned.

Other repo assets

.env.example contains names only, never credentials. LICENSE requires a deliberate team choice. GitHub Actions must run typecheck, focused tests and build. Screenshots belong in docs/assets/ with descriptive names and alt text in Markdown. Avoid personal data from third-party sites.

14 Demo script, Q&A; and evidence checklist

Two-minute narrative

Time	Action / words
0:00-0:20	Show the controlled flawed site and explain the goal: 'find the contact form.' A page-only scan lacks the full route.
0:20-0:50	Submit URL + goal in AccessPath. Show queued/running state and the TinyFish-discovered route. If live execution is slow, use a real precomputed scan and say so.
0:50-1:25	Open report. Show two confirmed axe-core findings with page, element, rule ID and screenshot. Mention manual-review boundary.
1:25-1:45	Show corrected fixture and second scan. Compare the same rules; do not claim all accessibility issues are solved.
1:45-2:00	Explain worker queue, owner isolation and what is measured or still planned; close with GitHub/live link.

Likely judge questions - concise answers

- **Why TinyFish?** It navigates a live website from a natural-language goal; our report depends on the route it discovers.
- **What does AI do?** Goal interpretation/navigation. axe-core, not AI, creates rule-based accessibility findings.
- **Can you certify WCAG?** No. Automated findings are a starting point; keyboard, assistive-technology and usability checks still need people.
- **How do you scale?** Queue/worker separation and bounded concurrency. We will cite measured throughput only after load testing.
- **What about malicious URLs?** Public HTTPS validation, redirect checks, quotas, private worker and time/resource bounds; share which controls are actually deployed.
- **What about cost?** Bounded page/session/time limits and daily caps; measured per-scan cost will be added after live runs.

Proof folder for the teammate

Ask developers for: current commit SHA, deployed URL, TinyFish run ID, real screenshots, actual axe rule IDs, CI result, two-user isolation test result, mobile view, and a dated list of incomplete features. Store only sanitized evidence in the repo.

15 Status ledger and handoff protocol

Update this table with the development team at every gate. This is the most important protection against a polished but inaccurate pitch.

Item	Baseline status - 17 Oct	Proof required before calling it built
AccessPath code	Planned	AccessPath repository commit and working app.
TinyFish Agent	Planned	Actual run ID, route result and capture.
TinyFish Browser + axe	Planned	Session/CDP run, raw findings and cleanup.
Multi-user isolation	Planned	Auth/RLS tests using two accounts.
Production deployment	Planned	Vercel + Railway URLs/status and a real production scan.
Impact or speed metrics	Unmeasured	Defined method, sample size and recorded results.
Older Resolve prototype	Separate project	Do not present it as AccessPath implementation.

What teammate can complete without interruption

- Draft the entire deck using the slide brief, with concept labels where screens are still pending.
- Draft README and docs with exact setup commands marked 'verify after Gate A/D'.
- Create the architecture diagram, source list, speaker notes, and Q&A; appendix.
- Prepare a final reconciliation checklist; request only factual build outputs from developers.

Single final reconciliation pass

Before submission: inspect the repository and live site; replace every placeholder; run README setup in a clean environment if possible; verify all URLs, commands, env variable names, screenshots, issue counts, test claims, and slide status wording; export the PPT only after those checks. Submit the GitHub and PPT before 18 Oct 2026, 11:59 PM IST.

16 Primary sources and verification notes

These are direct references for facts in this handoff. Product decisions are the team's proposal; source links verify vendor/event capabilities and the accessibility boundary. Accessed 17 October 2026 unless otherwise stated.

Source	What it supports
AZINHACK 2026 event	Schedule, team size, submission topics, GitHub/README requirement, open innovation, TinyFish requirement.
TinyFish Agent API	Goal-based run endpoints, structured output, async run ID, polling and cancellation.
TinyFish Browser API	Browser session create/delete, CDP URL and session timing notes.
axe-core	Automated accessibility rule engine and result context.
W3C WAI evaluation	Need for human evaluation; automated tool is not full conformance proof.
Vercel DDoS mitigation	Edge mitigation capability, not a guarantee of attack immunity.
Supabase RLS	Per-row data access controls.
Supabase Storage security	Private evidence and policy design.
Railway private networking	Private service-network design considerations.

Remaining unknowns that the team must fill from the actual build

Official team name and member links; final repository and deployment URLs; verified TinyFish plan/quota and spending limit; exact environment variable names; license; measured scan latency/cost; implemented security controls; actual finding rule IDs and screenshots; whether the event submission form imposes a file format or size limit. These cannot be responsibly invented in advance.

Final editorial rule: The event asks for a clear proposal and project repository. Present the proposed architecture confidently, but label build status precisely. A smaller, working and honestly documented system is a stronger submission than an unverified claim of perfect scalability, security or accessibility.